



Manual Técnico

Sistema de Gestión de Inventario
«Librería El Saber»

Aprendices

Carlos Iván Perdomo Perdomo

Luz Darlys González Torres

Instructor

Ingeniero Jairo Antonio Muñoz Arango

Programa de formación

Tecnólogo en Análisis y Desarrollo de Software (ADSO) — Ficha 3118323

Centro de formación

Centro Agropecuario La Granja — SENA Regional Tolima

Control del documento

Versión	Fecha	Descripción del cambio	Elaborado por
1.0	Agosto de 2026	Versión inicial para la sustentación del proyecto final	Carlos I. Perdomo · Luz D. González

Contenido

1. Introducción, propósito y glosario
2. Descripción del negocio y del sistema
3. Tabla de requerimientos funcionales y no funcionales
4. Historias de usuario y criterios de aceptación
5. Arquitectura del sistema
6. Diagrama de casos de uso (UML)
7. Diagrama de clases (UML)
8. Diagrama de paquetes (UML)
9. Diagrama de secuencia — registrar venta (UML)
10. Diagrama de despliegue (UML)
11. Modelo entidad–relación (MER)
12. Modelo relacional
13. Diccionario de datos (tablas descriptivas)
14. API REST — catálogo de endpoints
15. Seguridad del sistema
16. Documentación del código fuente
17. Pruebas automatizadas
18. Despliegue en producción
19. Estructura de carpetas del proyecto
20. Convenciones y elementos avanzados de la BD

1. Introducción, propósito y glosario

Propósito. Este manual recopila la arquitectura y la lógica interna del **SGI — Sistema de Gestión de Inventario «Librería El Saber»**: sus requerimientos, historias de usuario, diagramas UML, modelos de datos, API, seguridad y documentación del código. Su audiencia son desarrolladores, instructores técnicos y el jurado evaluador del SENA.

Alcance. Cubre el sistema completo: frontend (`cliente/`), backend (`servidor/`) y base de datos (`base_datos/`), tanto en ejecución local como en el despliegue de producción en Railway.

1.1 Glosario de términos

Término	Significado en este proyecto
SPA	Single Page Application: la interfaz carga una sola vez y navega sin recargar la página (React Router).
API REST	Estilo de comunicación HTTP donde el frontend consume recursos del backend en JSON (GET/POST/PUT/PATCH/DELETE).

JWT	JSON Web Token: «carné digital» firmado que identifica al usuario en cada petición (HS256, 8 horas).
RBAC	Role-Based Access Control: los permisos se asignan a roles (Administrador=1, Vendedor=2), no a personas.
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre una entidad.
Kardex	Registro histórico contable de entradas y salidas de mercancía (tabla <code>mdc_movimientos</code>).
POS	Point of Sale (punto de venta): la «caja registradora» digital del sistema.
ACID	Atomicidad, Consistencia, Aislamiento y Durabilidad: garantías de las transacciones (motor InnoDB).
FK / PK	Foreign Key (llave foránea) / Primary Key (llave primaria).
3FN	Tercera Forma Normal: nivel de normalización de la base de datos (sin redundancias).
Middleware	Función de Express que intercepta la petición antes del controlador (seguridad, validación, errores).
Pool de conexiones	Grupo de conexiones MySQL reutilizables (límite: 10) administrado por <code>mysql2</code> .

2. Descripción del negocio y del sistema

La idea de negocio. La «Librería El Saber» es una librería que llevaba el control de su inventario y sus ventas en cuadernos y hojas de cálculo, con los problemas típicos de ese método: no saber cuántos ejemplares quedan realmente, enterarse tarde de que un título se agotó, no tener trazabilidad de qué proveedor suministró cada lote ni a qué costo, y no poder consultar cifras del negocio sin contar a mano.

La solución. El SGI centraliza toda la operación en una aplicación web:

- Catálogo de libros con portada, autor, categoría, precio y control de stock con alertas de mínimos.
- Kardex de movimientos: cada entrada (compra a proveedor, con costo) y salida queda registrada con responsable, stock anterior y stock nuevo.
- Punto de venta con carrito, descuento porcentual, métodos de pago, ticket PDF y descuento automático de stock dentro de una transacción.
- Historial de ventas con filtros, exportación a Excel y anulación con reversa automática del inventario.
- Gestión de clientes, proveedores, autores, categorías y usuarios con control de acceso por roles.
- Dashboard gerencial con KPIs, tendencias y rankings para la toma de decisiones.

Ficha técnica	Detalle
Frontend	React 19 · Vite 7 · Bootstrap 5 · React Router DOM 7 · Axios · react-hook-form · Recharts · jsPDF · xlsx
Backend	Node.js ≥ 20 · Express 5 · mysql2 · jsonwebtoken · bcryptjs · Helmet · express-rate-limit · Multer + Cloudinary · compression · morgan · dotenv

Base de datos	MySQL 8 · esquema <code>inventario_libreria</code> · 10 tablas (prefijo <code>mdc_</code>) · InnoDB · utf8mb4 · 3FN · 3 vistas · índices de rendimiento
Pruebas	Jest 30 + Supertest (8 suites sobre la API real)
Producción	Railway (frontend + backend + MySQL) · Cloudinary (imágenes) · PM2 como gestor alternativo
Repositorio	github.com/cperdomope/SGI-Libreria-el-Saber (rama main)

3. Tabla de requerimientos

3.1 Requerimientos funcionales

Código	Requerimiento funcional	HU asociadas	Prioridad
RF-01	El sistema debe autenticar usuarios con correo y contraseña y emitir un token JWT con id, rol y nombre.	HU-01	Alta
RF-02	Debe bloquear la cuenta 3 minutos tras 3 intentos fallidos consecutivos y limitar a 10 intentos por IP cada 15 minutos.	HU-02	Alta
RF-03	Debe permitir cerrar la sesión eliminando el token y bloqueando el acceso a rutas protegidas.	HU-03	Alta
RF-04	Debe permitir a cada usuario cambiar su contraseña verificando la contraseña actual (mínimo 8 caracteres).	HU-04	Media
RF-05	Debe presentar al Administrador un dashboard con ventas del día, semana y mes (con % de crecimiento), alertas de stock y métricas de catálogo, inventario valorizado y clientes.	HU-05, HU-08	Alta
RF-06	Debe graficar ventas de los últimos 6 meses, ventas por día de la semana y distribución de libros por categoría.	HU-06	Media
RF-07	Debe calcular el top 5 de libros más vendidos y de mejores clientes, excluyendo ventas anuladas.	HU-07	Media
RF-08	Debe listar el inventario con portada, ISBN, autor, categoría, precio y estado visual del stock, y permitir búsqueda instantánea por título, autor o ISBN.	HU-09, HU-10	Alta
RF-09	Debe permitir al Administrador crear, editar y eliminar libros, con carga opcional de portada (JPG/PNG/WebP, máx. 2 MB) validada por extensión y tipo MIME.	HU-11 a HU-13	Alta
RF-10	Debe registrar ENTRADAS de inventario exigiendo proveedor y costo de compra, actualizando el stock automáticamente.	HU-14	Alta
RF-11	Debe registrar SALIDAS de inventario impidiendo que el stock quede negativo.	HU-15	Alta
RF-12	Debe conservar el Kardex completo: tipo, cantidad, stock anterior y nuevo, usuario responsable, proveedor, costo y fecha.	HU-16	Alta
RF-13	Debe registrar ventas con carrito multiartículo, cliente, método de pago (Efectivo, Tarjeta, Transferencia, Mixto) y descuento automático del stock en una transacción.	HU-17	Alta
RF-14	Debe aceptar un descuento porcentual (0–100 %) mostrando subtotal, descuento y total, validados nuevamente en el servidor.	HU-18	Media
RF-15	Debe recalcular el total de la venta en el servidor y rechazarla si no coincide con el enviado (control anti-fraude).	HU-17	Alta
RF-16	Debe listar el historial de ventas con paginación en el servidor, filtro por rango de fechas y búsqueda por cliente.	HU-19, HU-20	Alta

RF-17	Debe permitir al Administrador anular una venta: cambia el estado a «Anulada», devuelve el stock y registra la reversa en el Kardex, sin borrar el registro.	HU-21	Alta
RF-18	Debe generar el ticket de una venta en PDF con formato POS de 80 mm.	HU-22	Media
RF-19	Debe exportar a Excel (.xlsx) el historial de ventas filtrado y el inventario.	HU-23	Baja
RF-20	Debe gestionar clientes con documento único y tipos CC, NIT, CE y Pasaporte; Vendedor puede ver y crear, solo Administrador edita o elimina.	HU-24 a HU-28	Alta
RF-21	Debe gestionar proveedores (solo Administrador) con empresa, NIT, contacto y datos de contacto validados.	HU-29	Alta
RF-22	Debe gestionar autores y categorías, impidiendo eliminar los que tengan libros asociados y evitando categorías duplicadas.	HU-30, HU-31	Media
RF-23	Debe gestionar usuarios: crear con rol, editar datos, activar/desactivar (borrado lógico) sin permitir la autodesactivación del Administrador.	HU-32 a HU-35	Alta
RF-24	Debe aplicar control de acceso por roles en cada endpoint del backend y en la interfaz (menús y botones según permisos).	Transversal	Alta
RF-25	Debe exponer un health check público (GET /) que verifique servidor y conexión real a la BD.	Transversal	Media

3.2 Requerimientos no funcionales

Código	Categoría	Requerimiento no funcional
RNF-01	Seguridad	Las contraseñas se almacenan únicamente como hash bcrypt (10 salt rounds); nunca en texto plano ni expuestas por la API.
RNF-02	Seguridad	Las sesiones usan JWT firmado con HS256 y secreto de entorno (JWT_SECRET), con expiración de 8 horas y emisor sgi-libreria .
RNF-03	Seguridad	Rate limiting por IP: 10 intentos fallidos/15 min en login y 500 peticiones/15 min en la API general.
RNF-04	Seguridad	Cabeceras HTTP de seguridad con Helmet (CSP, HSTS, X-Frame-Options, nosniff) y CORS restringido a orígenes autorizados.
RNF-05	Integridad	Ventas, anulaciones y movimientos se ejecutan en transacciones ACID (InnoDB) con bloqueo de filas SELECT ... FOR UPDATE para impedir condiciones de carrera y stock negativo.
RNF-06	Rendimiento	Pool de 10 conexiones MySQL, índices sobre las columnas de consulta frecuente, consultas del dashboard en paralelo (Promise.all) con caché de 60 s, compresión gzip y code-splitting del frontend.
RNF-07	Usabilidad	Interfaz responsive (celular, tablet y PC), en español, con mensajes de error claros y validación doble (frontend y backend).
RNF-08	Disponibilidad	Apagado ordenado (graceful shutdown) ante SIGTERM/SIGINT, reinicio automático (Railway/PM2, límite 200 MB RAM) y health check monitorizable.
RNF-09	Mantenibilidad	Arquitectura por capas (rutas → middlewares → controladores → BD), principio DRY (middlewares, utilidades y hooks reutilizables), código comentado en español y ESLint en el frontend.

RNF-10	Escalabilidad	Paginación en el servidor (límite 100 por página), payload JSON limitado a 100 KB e imágenes servidas desde CDN (Cloudinary).
RNF-11	Portabilidad	Funciona en navegadores modernos sin instalación; el backend corre en cualquier entorno con Node 20+ y MySQL 8 (local o nube).
RNF-12	Confidencialidad	Los secretos (BD, JWT, Cloudinary) viven en variables de entorno (<code>.env</code>) excluido de Git; panel de Railway en producción).

4. Historias de usuario y criterios de aceptación

El proyecto se construyó a partir de **35 historias de usuario** en formato «*Como [rol], quiero [acción], para [beneficio]*», agrupadas en 10 módulos. Estas historias y sus criterios están también embebidos en la aplicación (botón «Documentación del Proyecto» del login).

4.1 Módulo: Autenticación

ID	Historia	Prioridad	Criterios de aceptación
HU-01	Iniciar sesión. Como usuario (Administrador o Vendedor), quiero iniciar sesión con mi correo y contraseña, para acceder a las funcionalidades según mi rol.	Alta	- Formulario con correo y contraseña; campos vacíos no se envían. - Con datos correctos redirige según el rol. - Con datos incorrectos muestra un error claro. - La sesión persiste al recargar (localStorage).
HU-02	Bloqueo por intentos fallidos. Como sistema, quiero bloquear la cuenta tras 3 intentos fallidos, para proteger las cuentas.	Alta	3 intentos fallidos → bloqueo de 3 minutos. - Barra de progreso con los intentos restantes. - Mensaje claro durante el bloqueo; se levanta solo.
HU-03	Cerrar sesión. Como usuario autenticado, quiero cerrar sesión desde la barra de navegación, para proteger mi cuenta.	Alta	- Opción visible en el menú de usuario, con confirmación. - Elimina la sesión y redirige al login. - Tras salir no se accede a páginas protegidas.
HU-04	Cambiar contraseña. Como usuario autenticado, quiero cambiar mi contraseña desde un modal, para mantener segura mi cuenta.	Media	- Pide contraseña actual, nueva (mín. 8) y confirmación. - Errores claros si la actual falla o no coinciden. - Éxito → mensaje y cierre del modal.

4.2 Módulo: Dashboard

ID	Historia	Prioridad	Criterios de aceptación
HU-05	Dashboard con estadísticas. Como Administrador, quiero un panel con ventas del día/semana/mes, alertas de stock y métricas de libros, inventario y clientes, para ver el estado del negocio.	Alta	- Solo Administradores; el Vendedor es redirigido a ventas. 4 tarjetas KPI + 3 métricas secundarias. - Los datos cargan automáticamente al entrar.
HU-06	Gráficas. Como Administrador, quiero una gráfica de área (6 meses) y una dona por categorías, para analizar ventas y catálogo.	Media	- Área con ventas e ingresos de 6 meses; dona por categoría. - Responsive; datos del servidor en tiempo real.

HU-07	Tops. Como Administrador, quiero ver los 5 libros más vendidos y los 5 mejores clientes.	Media	• Top de libros con ingresos generados. • Top de clientes por total gastado.
HU-08	Stock bajo. Como Administrador, quiero una tabla de libros con stock $\leq$ mínimo, para saber qué reabastecer.	Alta	• Tabla con título, stock actual y mínimo. • Enlace directo al inventario.

4.3 Módulo: Inventario

ID	Historia	Prioridad	Criterios de aceptación
HU-09	Ver libros. Como usuario, quiero ver todos los libros con imagen, título, autor, categoría, precio y stock.	Alta	• Tabla con portada, ISBN, título, autor, categoría, precio y stock. • Badge de estado: Disponible / Stock Bajo / Agotado. • Scroll horizontal en celulares.
HU-10	Buscar libros. Como usuario, quiero buscar por título, autor o ISBN.	Alta	• Filtrado instantáneo sin petición al servidor. • Mensaje cuando no hay resultados.
HU-11	Crear libro. Como Administrador, quiero registrar un libro con sus datos y portada.	Alta	• Botón visible solo para Administrador. • Portada JPG/PNG/WebP máx. 2 MB. • Validaciones antes de enviar; aparece sin recargar.
HU-12	Editar libro. Como Administrador, quiero modificar los datos de un libro.	Alta	• Formulario precargado; puede cambiar la portada. • Cambios inmediatos en la tabla.
HU-13	Eliminar libro. Como Administrador, quiero eliminar un libro del inventario.	Media	• Confirmación previa; borra también la portada. • No se elimina si tiene historial (FK RESTRICT).

4.4 Módulo: Movimientos (Kardex)

ID	Historia	Prioridad	Criterios de aceptación
HU-14	Registrar entrada. Como Administrador, quiero registrar entradas con libro, cantidad, proveedor y costo, para que el stock suba automáticamente.	Alta	• Proveedor y costo obligatorios en ENTRADA. • Stock se actualiza y el movimiento queda en el panel.
HU-15	Registrar salida. Como Administrador, quiero registrar salidas por motivos distintos a venta.	Alta	• No permite sacar más del stock disponible. • Stock se descuenta automáticamente.
HU-16	Historial Kardex. Como Administrador, quiero ver el historial completo de movimientos.	Alta	• Muestra libro, tipo, cantidad, stock anterior/nuevo, proveedor, costo, usuario y fecha. • Se actualiza al registrar.

4.5 Módulo: Ventas

ID	Historia	Prioridad	Criterios de aceptación
HU-17	Registrar venta. Como usuario, quiero buscar cliente, armar carrito, elegir pago y confirmar, para vender y descontar stock automáticamente.	Alta	- Carrito multiartículo con cantidades editables. - Métodos: Efectivo, Tarjeta, Transferencia, Mixto. - El backend recalcula y valida el total.
HU-18	Descuento. Como usuario, quiero aplicar un descuento porcentual (0–100 %).	Media	- Muestra subtotal, monto de descuento y total. - Validado también en el servidor; queda en BD, historial y PDF.
HU-19	Historial. Como usuario, quiero ver todas las ventas con su estado.	Alta	- Estados visuales Completada/Anulada; paginación de 10. - Acceso al detalle de cada venta.
HU-20	Filtros. Como usuario, quiero filtrar por fechas y buscar por cliente.	Media	- Campos desde/hasta y buscador combinables. - Sin recargar la página.
HU-21	Anular venta. Como Administrador, quiero anular una venta y que el stock se devuelva.	Alta	- Solo Administrador, con confirmación. - Devuelve stock, estado «Anulada», sin re-anulación.
HU-22	Ticket PDF. Como usuario, quiero descargar el ticket de una venta.	Media	- Formato POS 80 mm con cliente, productos y total. - Descarga inmediata.
HU-23	Exportar Excel. Como usuario, quiero exportar la lista filtrada de ventas.	Baja	Respetar los filtros activos; archivo .xlsx.

4.6 Módulos: Clientes, Proveedores, Autores, Categorías y Usuarios

ID	Historia	Prioridad	Criterios de aceptación
HU-24	Ver clientes. Como usuario, quiero ver los clientes con su documento y contacto.	Alta	Tabla con nombre, tipo y número de documento, teléfono y correo.
HU-25	Buscar clientes. Como usuario, quiero buscar por nombre o documento.	Alta	Filtrado instantáneo; mensaje sin resultados.
HU-26	Crear cliente. Como usuario, quiero registrar clientes con tipo de documento CC/NIT/CE/Pasaporte.	Alta	- Vendedor y Administrador pueden crear. - Documento único.
HU-27	Editar cliente. Como Administrador, quiero corregir los datos de un cliente.	Media	Solo Administrador; formulario precargado.
HU-28	Eliminar cliente. Como Administrador, quiero eliminar clientes.	Baja	Con confirmación; protegido si tiene ventas (FK RESTRICT).

HU-29	Gestionar proveedores. Como Administrador, quiero CRUD de proveedores (empresa, NIT, contacto...).	Alta	- Exclusivo del Administrador. - No se elimina si tiene movimientos; mensaje claro.
HU-30	Gestionar autores. Como Administrador, quiero CRUD de autores.	Media	- Todos consultan; solo Admin modifica. - No se elimina un autor con libros.
HU-31	Gestionar categorías. Como Administrador, quiero CRUD de categorías.	Media	Nombre único; no se elimina con libros asociados.
HU-32	Ver usuarios. Como Administrador, quiero ver los usuarios con rol, estado y último acceso.	Alta	Solo Administrador; badges de rol y estado.
HU-33	Crear usuario. Como Administrador, quiero crear cuentas con rol.	Alta	Email único; contraseña mínima de 8; queda activo.
HU-34	Editar usuario. Como Administrador, quiero editar nombre, correo y rol.	Media	La contraseña no se cambia desde aquí.
HU-35	Activar/desactivar usuario. Como Administrador, quiero desactivar cuentas sin borrarlas.	Alta	Inactivo no inicia sesión; sin autodesactivación.

5. Arquitectura del sistema

El SGI implementa una **arquitectura cliente-servidor de tres capas** con un servicio externo de almacenamiento de imágenes. El frontend es una SPA que consume la API REST; el backend organiza la lógica en capas (rutas → middlewares → controladores → acceso a datos) siguiendo el patrón MVC adaptado a API (las «vistas» son las respuestas JSON y la interfaz React).

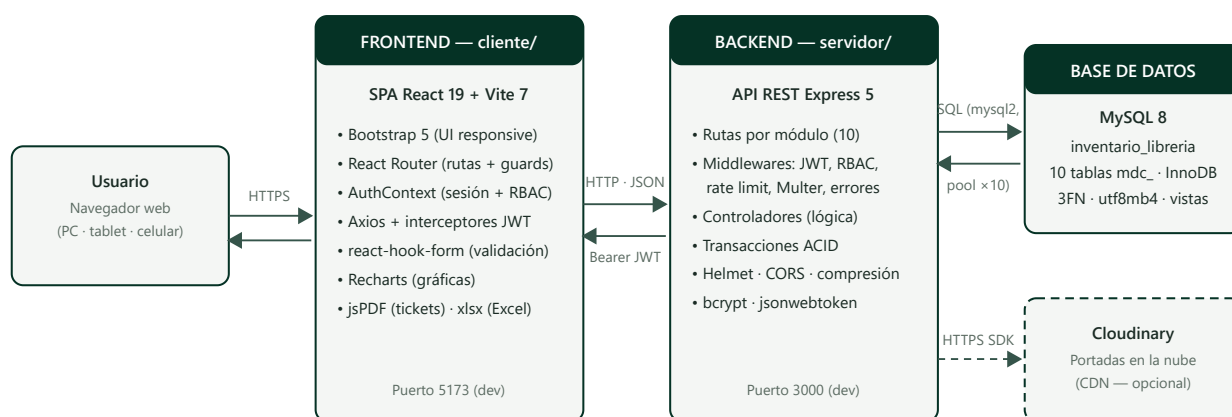


FIGURA 1. Arquitectura general del sistema: SPA + API REST + MySQL + Cloudinary.

5.1 Flujo de una petición (ciclo completo)

1. El usuario acciona la interfaz (por ejemplo, «Confirmar venta») en un componente React.
2. `services/api.js` (Axios) envía la petición; su **interceptor** adjunta automáticamente `Authorization: Bearer <JWT>`.
3. En el backend la petición atraviesa la cadena de middlewares globales de `app.js`: `helmet` → `compression` → `morgan` → `cors` → `static` → `express.json (100 KB)` → `rate limiter`.
4. El enrutador del módulo aplica los middlewares de la ruta: `verificarToken` (autenticación JWT), `verificarRol` (autorización RBAC) y `validarId` cuando hay parámetros.
5. El **controlador** valida los datos de negocio, ejecuta las consultas (transacción si aplica) sobre el **pool** de `config/db.js` y responde JSON.
6. Si algo falla, el **errorHandler** global traduce el error a una respuesta JSON estandarizada sin exponer detalles internos.
7. El interceptor de respuesta del frontend detecta un 401 (sesión vencida) y redirige al login; en caso normal, el componente actualiza la interfaz.

6. Diagrama de casos de uso (UML)

Los actores son **Vendedor** y **Administrador**. El Administrador *generaliza* al Vendedor: hereda todos sus casos de uso y añade los administrativos. Las relaciones «include» muestran comportamiento obligatorio incorporado por otro caso de uso.

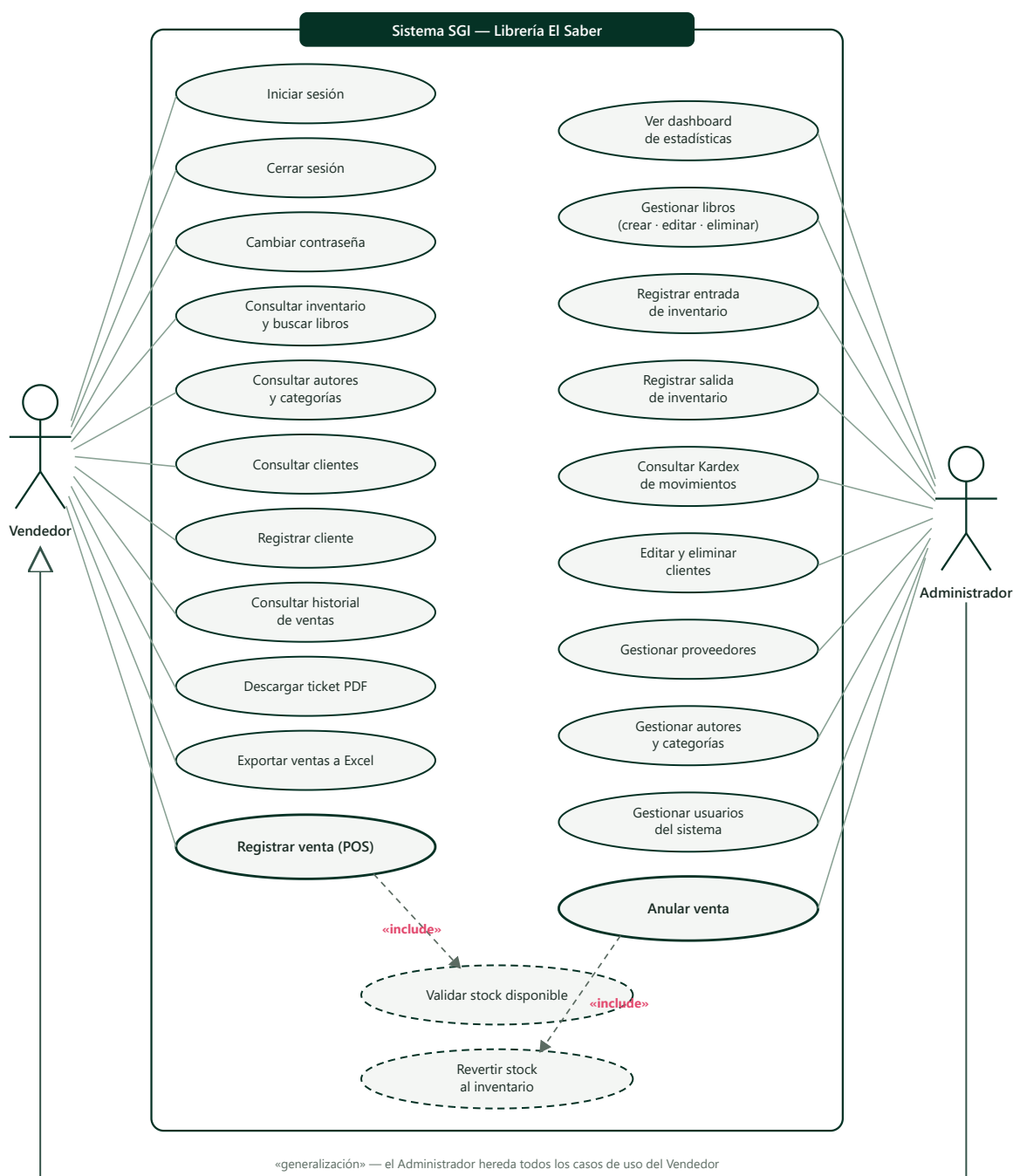


FIGURA 2. Diagrama de casos de uso del SGI. «Registrar venta» incluye la validación de stock (con bloqueo de fila) y «Anular venta» incluye la reversión del stock con registro en el Kardex.

7. Diagrama de clases (UML)

El diagrama modela las **entidades del dominio**. Los atributos provienen del esquema de la base de datos y los métodos de las operaciones expuestas por los controladores del backend. La relación Venta-DetalleVenta es una **composición** (el detalle no existe sin su venta; en BD: `ON DELETE CASCADE`).

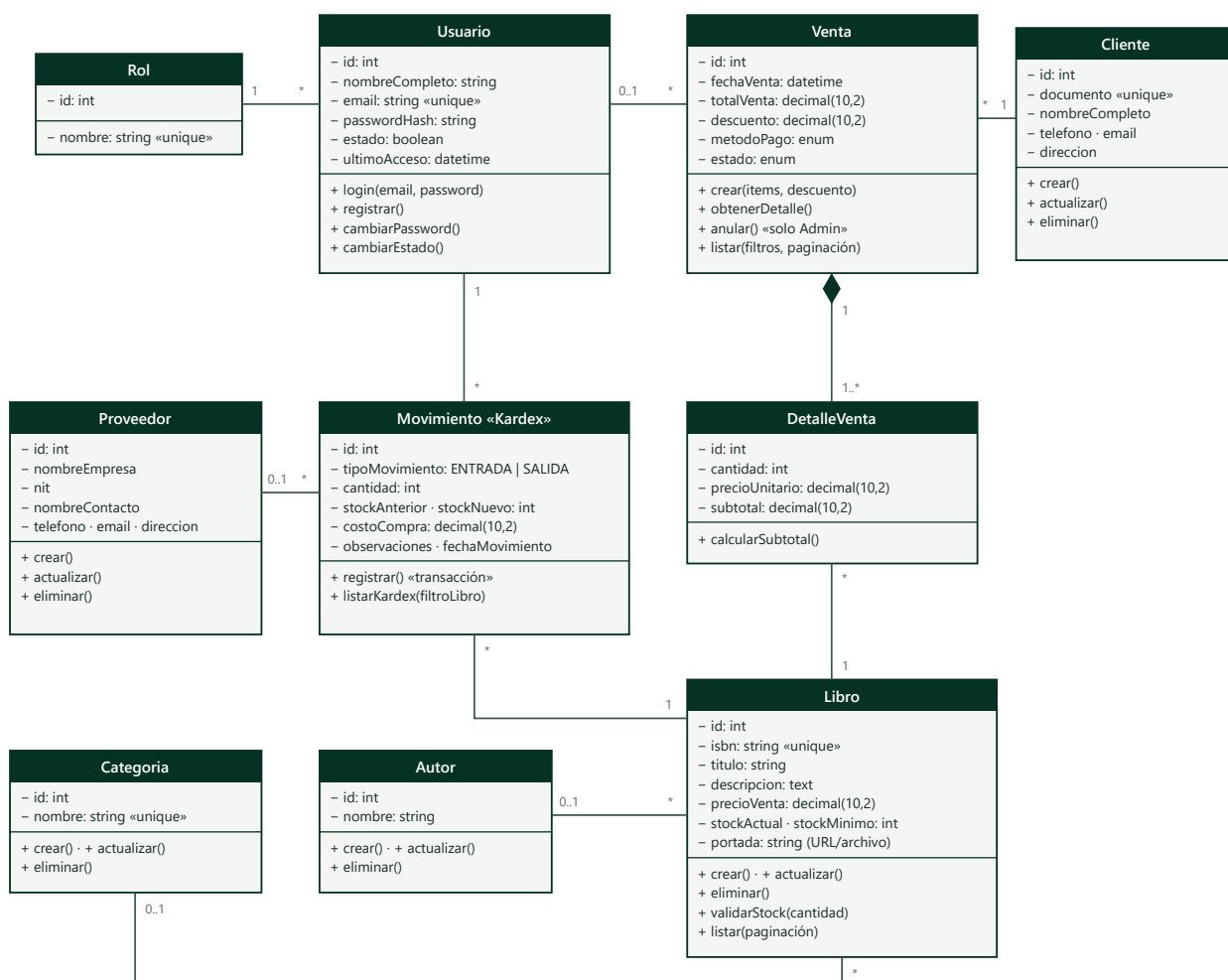


FIGURA 3. Diagrama de clases del dominio con atributos, operaciones y multiplicidades. El rombo relleno indica composición (una venta se compone de sus líneas de detalle).

8. Diagrama de paquetes (UML)

Organización física del código en paquetes y sus dependencias. La única «frontera» entre el cliente y el servidor es la API REST (HTTP/JSON); el servidor es el único que dialoga con la base de datos.

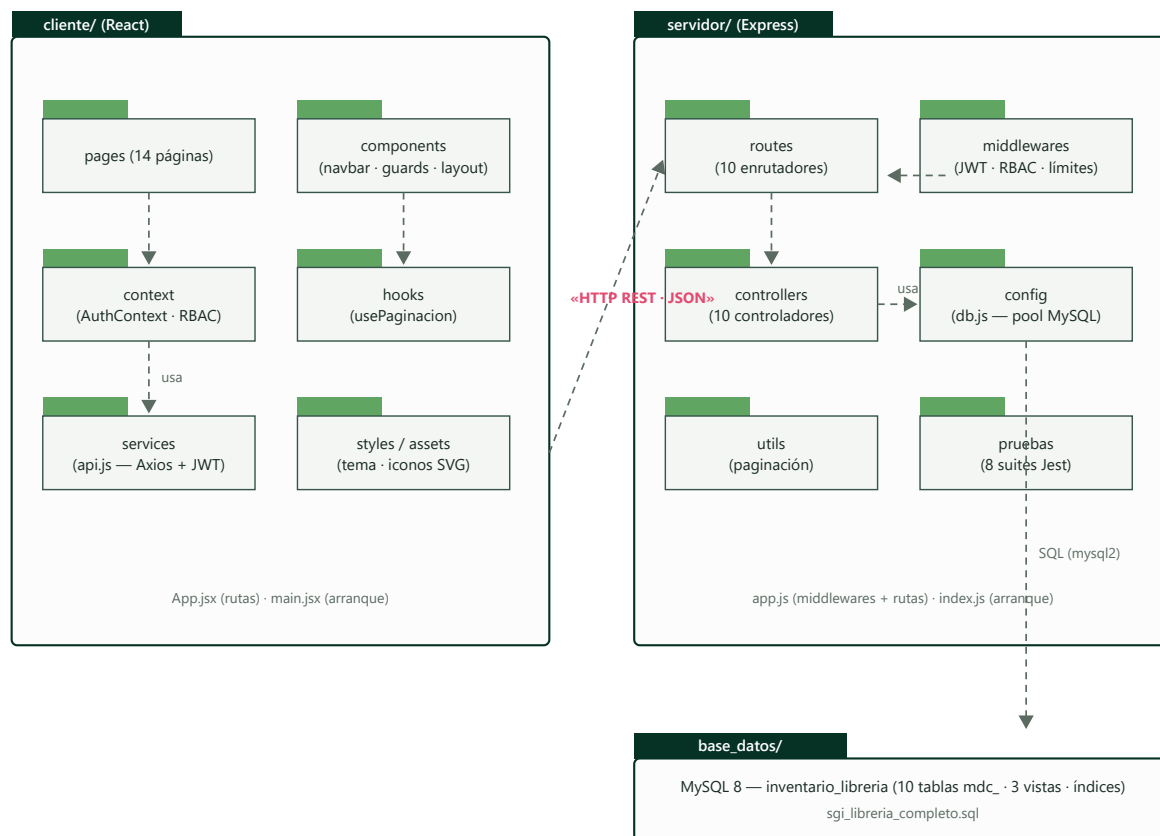
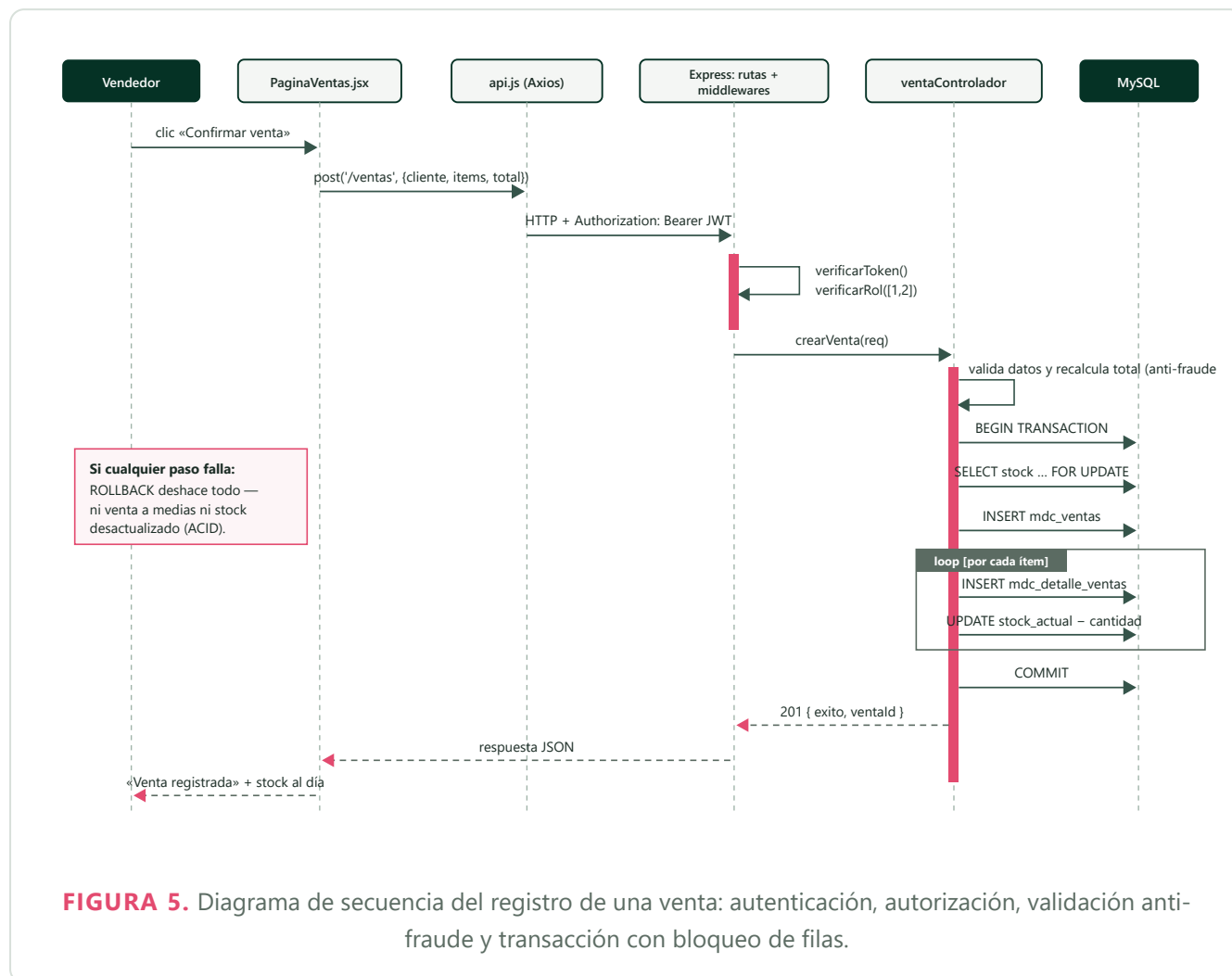


FIGURA 4. Diagrama de paquetes: organización del código y dependencias. El paquete **services** del cliente es el único punto de comunicación con el paquete **routes** del servidor.

9. Diagrama de secuencia — registrar venta (UML)

Interacción completa del caso de uso más crítico del sistema, incluida la transacción ACID con bloqueo de filas que garantiza la consistencia del stock.



10. Diagrama de despliegue (UML)

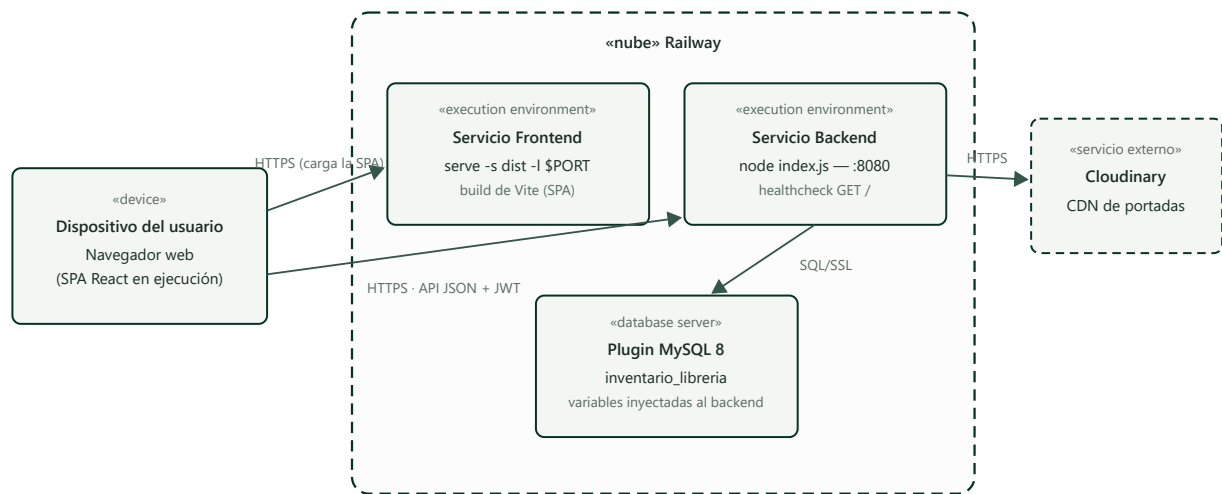


FIGURA 6. Diagrama de despliegue en producción: tres servicios en Railway y Cloudinary como CDN de imágenes. El proxy de Railway exige `trust proxy` en Express para el rate limiting.

11. Modelo entidad–relación (MER)

Modelo conceptual en notación de Chen: las entidades (rectángulos) se asocian mediante relaciones (rombos) con sus cardinalidades. **DETALLE_VENTA** es la entidad asociativa que resuelve la relación muchos-a-muchos entre **VENTA** y **LIBRO**. Los atributos de cada entidad se detallan en el diccionario de datos (sección 13).

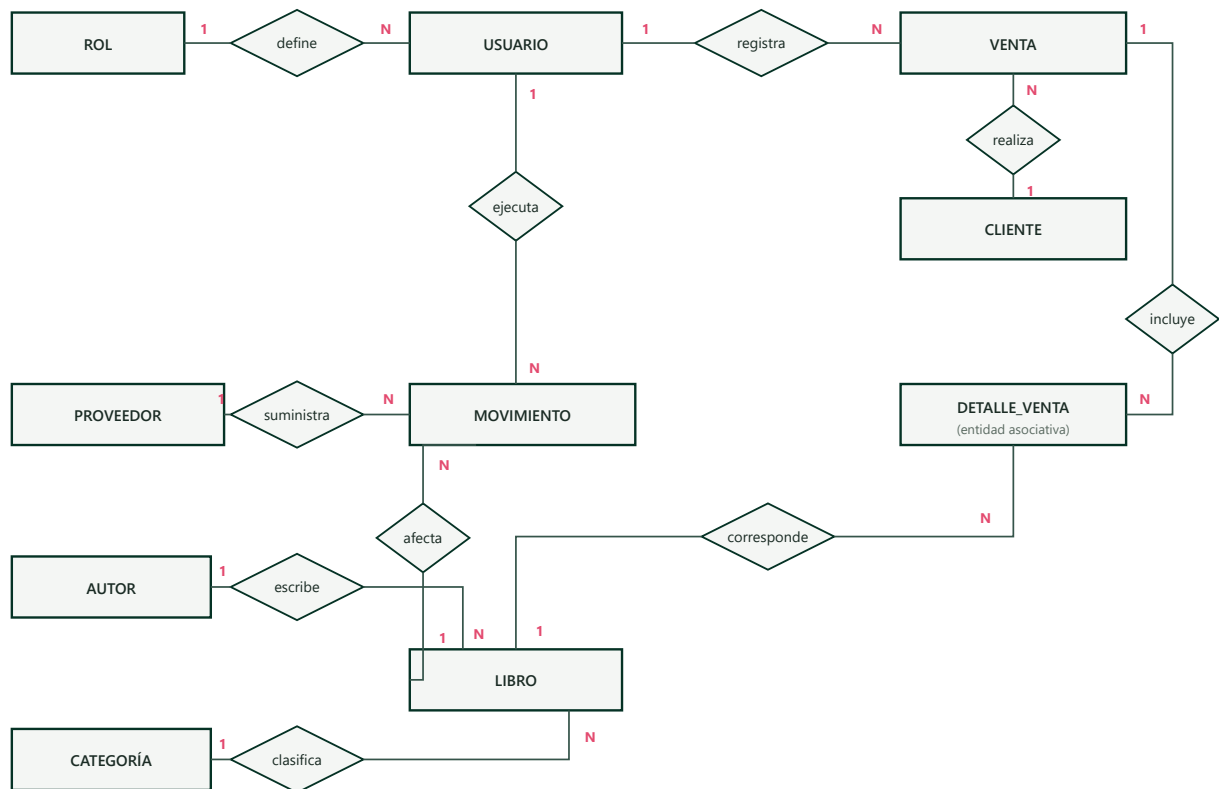


FIGURA 7. Modelo entidad–relación (notación de Chen). Lecturas: un ROL define N USUARIOS; un CLIENTE realiza N VENTAS; una VENTA incluye N líneas de DETALLE y cada línea corresponde a 1 LIBRO; un PROVEEDOR suministra N MOVIMIENTOS que afectan 1 LIBRO cada uno.

12. Modelo relacional

Modelo lógico implementado en MySQL: las 10 tablas con sus llaves primarias (PK), foráneas (FK) y las relaciones 1—∞. Corresponde exactamente al script `base_datos/sgi_libreria_completo.sql`.

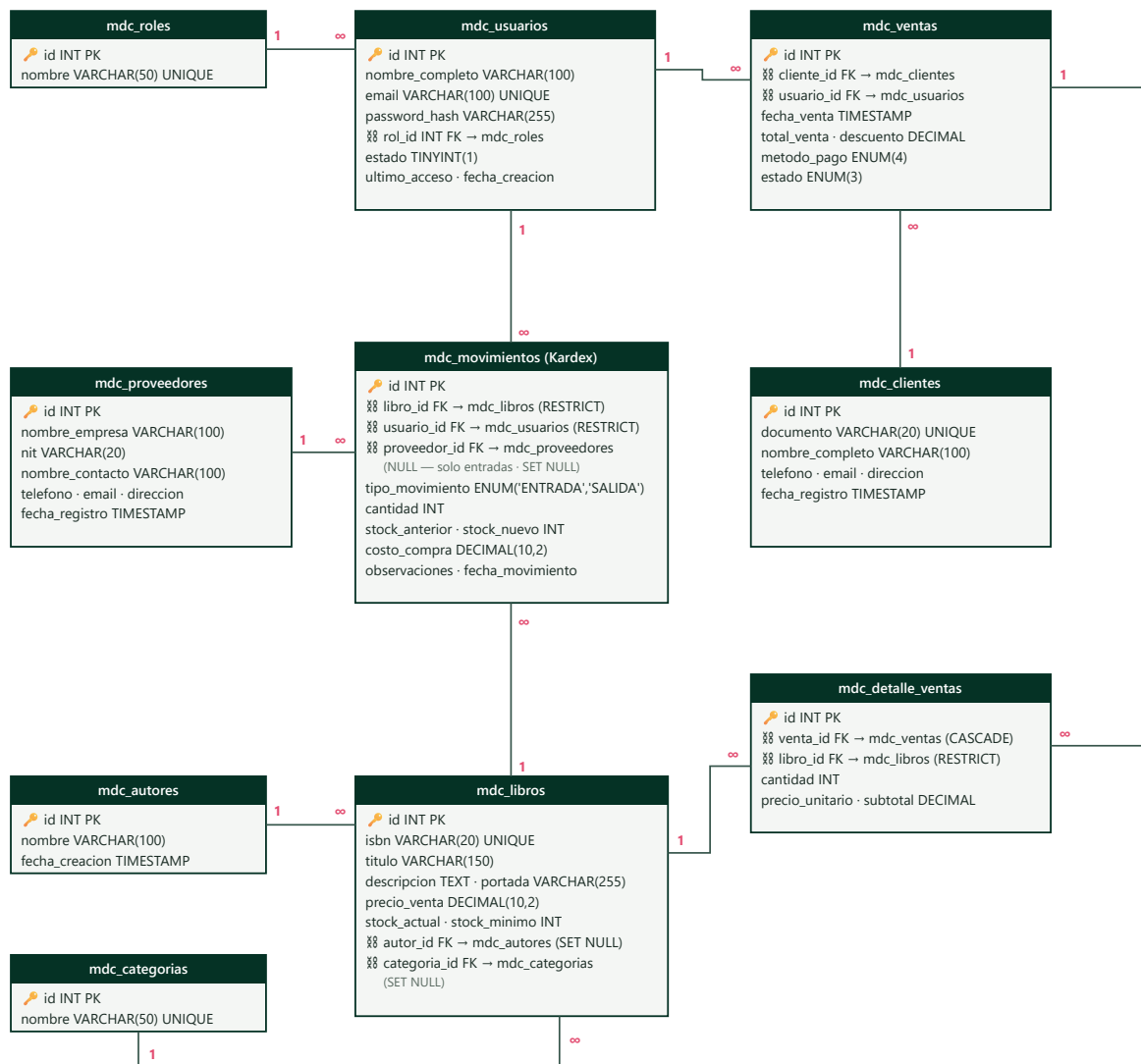


FIGURA 8. Modelo relacional de `inventario_libreria` (1—∞). Las políticas de borrado protegen el historial: RESTRICT impide borrar registros referenciados, CASCADE limpia los detalles de una venta eliminada y SET NULL conserva libros y movimientos aunque se borre el catálogo relacionado.

13. Diccionario de datos (tablas descriptivas)

Descripción campo a campo de las 10 tablas del esquema `inventario_libreria` (motor InnoDB, charset utf8mb4, collation utf8mb4_unicode_ci).

13.1 mdc_roles — Roles del sistema RBAC

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del rol (1=Administrador, 2=Vendedor)
nombre	VARCHAR(50)	No	UNIQUE	Nombre del rol

13.2 mdc_usuarios — Usuarios del sistema

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del usuario
nombre_completo	VARCHAR(100)	No	—	Nombre del empleado
email	VARCHAR(100)	No	UNIQUE	Correo de inicio de sesión (se guarda en minúsculas)
password_hash	VARCHAR(255)	No	—	Hash bcrypt de la contraseña (nunca texto plano)
rol_id	INT	No	FK → mdc_roles.id (ON UPDATE CASCADE)	Rol del usuario
estado	TINYINT(1)	No (def. 1)	—	1=Activo, 0=Inactivo (borrado lógico)
ultimo_acceso	TIMESTAMP	Sí	—	Fecha/hora del último login (se actualiza en cada acceso)
fecha_creacion	TIMESTAMP	No (def. NOW)	—	Fecha de creación de la cuenta

13.3 mdc_autores — Catálogo de autores

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del autor
nombre	VARCHAR(100)	No	—	Nombre del autor
fecha_creacion	TIMESTAMP	No (def. NOW)	—	Fecha de registro

13.4 mdc_categorias — Categorías de libros

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador de la categoría
nombre	VARCHAR(50)	No	UNIQUE	Nombre (no admite duplicados)
fecha_creacion	TIMESTAMP	No (def. NOW)	—	Fecha de registro

13.5 mdc_libros — Inventario principal

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del libro
isbn	VARCHAR(20)	Sí	UNIQUE	Código ISBN-13 del libro
portada	VARCHAR(255)	Sí	—	Nombre de archivo local o URL de Cloudinary de la portada
titulo	VARCHAR(150)	No	INDEX	Título del libro
descripcion	TEXT	Sí	—	Sinopsis / descripción comercial
precio_venta	DECIMAL(10,2)	No	—	Precio en COP (DECIMAL evita errores de redondeo del FLOAT)
stock_actual	INT	No (def. 0)	INDEX compuesto	Unidades disponibles; solo cambia por movimientos/ventas
stock_minimo	INT	No (def. 5)	INDEX compuesto	Umbral que dispara la alerta de reabastecimiento
autor_id	INT	Sí	FK → mdc_autores.id (ON DELETE SET NULL)	Autor del libro
categoria_id	INT	Sí	FK → mdc_categorias.id (ON DELETE SET NULL)	Categoría del libro
fecha_creacion	TIMESTAMP	No (def. NOW)	—	Fecha de registro

13.6 mdc_clientes — Clientes de la librería

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del cliente
documento	VARCHAR(20)	No	UNIQUE	CC, NIT, CE o pasaporte — impide clientes duplicados
nombre_completo	VARCHAR(100)	No	INDEX	Nombre o razón social

telefono	VARCHAR(20)	Sí	—	Teléfono de contacto
email	VARCHAR(100)	Sí	—	Correo del cliente
direccion	VARCHAR(200)	Sí	—	Dirección
fecha_registro	TIMESTAMP	No (def. NOW)	—	Fecha de registro

13.7 mdc_proveedores — Proveedores

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del proveedor
nombre_empresa	VARCHAR(100)	No	—	Razón social de la distribuidora/editorial
nit	VARCHAR(20)	Sí	—	NIT con dígito de verificación
nombre_contacto	VARCHAR(100)	Sí	—	Persona de contacto
telefono	VARCHAR(20)	Sí	—	Teléfono (validado a 10 dígitos en el backend)
email	VARCHAR(100)	Sí	—	Correo comercial
direccion	VARCHAR(200)	Sí	—	Dirección
fecha_registro	TIMESTAMP	No (def. NOW)	—	Fecha de registro

13.8 mdc_movimientos — Kardex de inventario

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador del movimiento
libro_id	INT	No	FK → mdc_libros.id (ON DELETE RESTRICT)	Libro afectado; no puede borrarse un libro con Kardex
usuario_id	INT	No	FK → mdc_usuarios.id (ON DELETE RESTRICT)	Usuario responsable (trazabilidad)
tipo_movimiento	ENUM('ENTRADA','SALIDA')	No	—	Naturaleza del movimiento
cantidad	INT	No	—	Unidades del movimiento (entero positivo)
fecha_movimiento	TIMESTAMP	No (def. NOW)	INDEX	Fecha/hora del movimiento

observaciones	TEXT	Sí	—	Motivo, número de factura, etc.
proveedor_id	INT	Sí	FK → mdc_proveedores.id (SET NULL / UPDATE CASCADE)	Proveedor que suministró (solo ENTRADA)
costo_compra	DECIMAL(10,2)	Sí	—	Costo unitario de compra (solo ENTRADA; 0 = donación)
stock_anterior	INT	Sí	—	«Foto» del stock antes del movimiento (auditoría)
stock_nuevo	INT	Sí	—	«Foto» del stock después del movimiento

13.9 mdc_ventas — Cabecera de facturas

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Número de la venta/factura
cliente_id	INT	No	FK → mdc_clientes.id (ON DELETE RESTRICT) · INDEX	Cliente que compró; protege el historial de facturación
usuario_id	INT	Sí	FK → mdc_usuarios.id (ON DELETE SET NULL)	Vendedor que registró la venta
fecha_venta	TIMESTAMP	No (def. NOW)	INDEX	Fecha/hora de la venta
total_venta	DECIMAL(10,2)	No	—	Total cobrado (subtotal, descuento, impuestos, validado en servidor)
descuento	DECIMAL(10,2)	No (def. 0)	—	Monto de descuento aplicado
metodo_pago	ENUM('Efectivo', 'Tarjeta', 'Transferencia', 'Mixto')	No (def. Efectivo)	—	Forma de pago

estado	ENUM('Completada','Anulada','Pendiente')	No (def. Completada)	INDEX	Estado; la anulación borra el registro
--------	--	----------------------	-------	--

13.10 mdc_detalle_ventas — Ítems de cada factura

Campo	Tipo	Nulo	Llave	Descripción
id	INT AUTO_INCREMENT	No	PK	Identificador de la línea
venta_id	INT	No	FK → mdc_ventas.id (ON DELETE CASCADE) · INDEX	Venta a la que pertenece; se elimina junto con su venta
libro_id	INT	No	FK → mdc_libros.id (ON DELETE RESTRICT) · INDEX	Libro vendido; impide borrar libros facturados
cantidad	INT	No	—	Unidades vendidas
precio_unitario	DECIMAL(10,2)	No	—	Precio al momento de la venta (histórico, aunque el precio cambie)
subtotal	DECIMAL(10,2)	No	—	cantidad × precio_unitario (recalculado en el servidor)



14. API REST — catálogo de endpoints

Todos los endpoints (salvo el login y el health check) exigen el encabezado `Authorization: Bearer <token>`. La columna «Acceso» refleja el middleware RBAC aplicado en el archivo de rutas correspondiente.

Método	Ruta	Función	Acceso
GET	/	Health check: estado del servidor y de la conexión a la BD	Público
POST	/api/auth/login	Iniciar sesión y emitir JWT (rate limit 10/15 min)	Público
POST	/api/auth/registro	Registrar un usuario	Admin
GET	/api/libros	Listar libros con autor y categoría (paginación opcional)	Admin y Vendedor
POST	/api/libros	Crear libro (multipart con portada opcional)	Admin
PUT	/api/libros/:id	Actualizar libro (reemplaza portada si se envía)	Admin
DELETE	/api/libros/:id	Eliminar libro (bloqueado si tiene historial)	Admin
GET	/api/movimientos	Historial del Kardex (filtro ?libro_id=)	Admin
POST	/api/movimientos	Registrar ENTRADA/SALIDA (transacción)	Admin
GET	/api/ventas	Listar ventas (filtros fecha/cliente + paginación servidor)	Admin y Vendedor
GET	/api/ventas/:id	Detalle completo de una venta con sus ítems	Admin y Vendedor
POST	/api/ventas	Crear venta (transacción + FOR UPDATE + validación de total)	Admin y Vendedor
PATCH	/api/ventas/:id/anular	Anular venta y revertir stock con registro en Kardex	Admin
GET	/api/clientes	Listar clientes (paginación opcional)	Admin y Vendedor
GET	/api/clientes/:id	Consultar un cliente	Admin y Vendedor
POST	/api/clientes	Crear cliente (documento único)	Admin y Vendedor
PUT	/api/clientes/:id	Actualizar cliente	Admin
DELETE	/api/clientes/:id	Eliminar cliente (bloqueado si tiene ventas)	Admin
GET	/api/dashboard	Estadísticas del negocio (15 consultas en paralelo, caché 60 s)	Admin
GET	/api/proveedores	Listar proveedores	Admin
POST	/api/proveedores	Crear proveedor	Admin

PUT	/api/proveedores/:id	Actualizar proveedor	Admin
DELETE	/api/proveedores/:id	Eliminar proveedor	Admin
GET	/api/autores	Listar autores	Admin y Vendedor
POST	/api/autores	Crear autor	Admin
PUT	/api/autores/:id	Actualizar autor	Admin
DELETE	/api/autores/:id	Eliminar autor (bloqueado si tiene libros)	Admin
GET	/api/categorias	Listar categorías	Admin y Vendedor
POST	/api/categorias	Crear categoría (nombre único)	Admin
PUT	/api/categorias/:id	Actualizar categoría	Admin
DELETE	/api/categorias/:id	Eliminar categoría (bloqueada si tiene libros)	Admin
GET	/api/usuarios	Listar usuarios (sin password_hash)	Admin
POST	/api/usuarios	Crear usuario con rol	Admin
PUT	/api/usuarios/:id	Actualizar nombre, email y rol	Admin
PATCH	/api/usuarios/:id/estado	Activar/desactivar (sin autodesactivación)	Admin
PATCH	/api/usuarios/cambiar-password	Cambiar contraseña propia (verifica la actual)	Usuario autenticado

15. Seguridad del sistema

La seguridad está construida en **capas** (defensa en profundidad); cada capa detiene un tipo distinto de ataque:

Capa	Mecanismo	Implementación
1. Red	CORS restringido	<code>app.js</code> : lista de orígenes permitidos (localhost + dominio de producción + <code>CORS_ORIGIN</code>)
2. Cabeceras	Helmet	CSP, HSTS, X-Frame-Options (anti-clickjacking), nosniff automáticos
3. Tasa de peticiones	express-rate-limit	Login: 10 intentos fallidos/15 min por IP · API: 500/15 min por IP · <code>trust proxy</code> para IP real tras Railway
4. Anti fuerza bruta por cuenta	Bloqueo temporal	<code>authControlador.js</code> : Map en memoria — 3 fallos → 3 min de bloqueo por email, con limpieza periódica
5. Autenticación	JWT HS256	<code>verificarToken.js</code> : valida firma y expiración (8 h); mensajes diferenciados <code>TOKEN_EXPIRED</code> / <code>TOKEN_INVALID</code>
6. Autorización	RBAC	<code>verificarRol.js</code> (factory): presets <code>soloAdministrador</code> y <code>administradorOVendedor</code> por ruta; espejo en la matriz <code>PERMISOS</code> del frontend (solo UX)

7. Datos de entrada	Validación doble	react-hook-form en el cliente + validaciones de negocio en cada controlador + <code>validarParametroId</code> para IDs de URL + JSON limitado a 100 KB
8. Contraseñas	bcrypt (10 rounds)	Hash con salt; <code>bcrypt.compare()</code> en login; password_hash jamás viaja en respuestas
9. Integridad transaccional	ACID + FOR UPDATE	Ventas, anulaciones y movimientos en transacciones; bloqueo de filas contra condiciones de carrera; total recalculado en servidor (anti-fraude)
10. Archivos	Multer con doble validación	Extensión + MIME real (JPG/PNG/WebP), límite 2 MB, nombres únicos; Cloudinary en producción
11. Secretos	Variables de entorno	<code>.env</code> fuera de Git; en producción los secretos viven en el panel de Railway
12. Errores	errorHandler global	Respuestas JSON estandarizadas; en producción no se exponen detalles internos ni stack traces

16. Documentación del código fuente

Bloques de código reales del proyecto con su explicación. Todo el código fuente está además comentado línea a línea en español dentro del repositorio.

16.1 cliente/index.html — punto de entrada de la SPA

Es el único HTML del proyecto: Vite inyecta aquí los scripts compilados y React monta toda la interfaz dentro de `<div id="root">`.

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>SGI - Libreria El Saber</title>
  </head>
  <body>
    <div id="root"></div>                                <!-- React monta aquí TODA la interfaz -->
    <script type="module" src="/src/main.jsx"></script>  <!-- Vite inyecta el bundle -->
  </body>
</html>
```

- `lang="es"` declara el idioma; `viewport` habilita el diseño responsive.
- `type="module"` permite usar imports de ES Modules directamente en el navegador.
- `main.jsx` importa Bootstrap → tema propio → estilos del login (el orden define la cascada CSS) y monta `<App />` con `createRoot`.

16.2 servidor/app.js — la cadena de middlewares

Configura la aplicación Express sin arrancarla (eso lo hace `index.js`); así las pruebas pueden importarla sin abrir puertos. El orden de los middlewares es deliberado:

```
app.set('trust proxy', 1); // IP real detrás del proxy de Railway
app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
app.use(compression()); // gzip: -70/80 % en respuestas JSON
app.use(morgan(...)); // logging de cada petición
app.use(cors({ origin: origenesPermitidos, credentials: true, ... }));
app.use('/uploads', express.static(...)); // portadas locales como URL pública
app.use(express.json({ limit: '100kb' })); // anti «JSON bomb»
app.use('/api', limiterAPI); // 500 peticiones/15 min por IP
app.use('/api/auth', rutasAuth); // ... y las otras 9 rutas de módulo
app.get('/', healthCheck); // prueba real: SELECT 1 a la BD
app.use(notFoundHandler); // 404 para rutas inexistentes
app.use(errorHandler); // red de seguridad global
```

16.3 servidor/config/db.js — pool de conexiones

```
const pool = mysql.createPool({
  host: process.env.DB_HOST, user: process.env.DB_USER,
  password: process.env.DB_PASSWORD, database: process.env.DB_NAME,
  charset: 'utf8mb4',
  waitForConnections: true, connectionLimit: 10, // 10 conexiones reutilizables
  idleTimeout: 60000,
  ...(usarSSL && { ssl: { rejectUnauthorized: false } }) // SSL solo en nube
});
module.exports = pool.promise(); // habilita async/await en los controladores
```

El archivo valida al arrancar que existan las variables obligatorias (*fail-fast*) y prueba la conexión sin tumbar el servidor, para que el health check pueda reportar el estado real.

16.4 servidor/middlewares/verificarToken.js — autenticación

```
const partes = req.headers.authorization.split(' '); // "Bearer eyJ..."
if (partes.length !== 2 || partes[0] !== 'Bearer') return res.status(401)...;
const datosUsuario = jwt.verify(partes[1], process.env.JWT_SECRET);
req.usuario = datosUsuario; // { id, rol, nombre, email } disponible para el controlador
next();
```

Diferencia los errores `TokenExpiredError` (sesión vencida) y `JsonWebTokenError` (token alterado) con códigos distintos que el frontend interpreta.

16.5 servidor/middlewares/verificarRol.js — autorización (factory)

```
const verificarRol = (rolesPermitidos) => (req, res, next) => {
  if (!req.usuario) return res.status(401).json({ codigo: 'NOT_AUTHENTICATED' });
  if (!rolesPermitidos.includes(req.usuario.rol))
    return res.status(403).json({ codigo: 'FORBIDDEN' }); // autenticado pero sin permiso
  next();
};
const soloAdministrador = verificarRol([ROLES.ADMINISTRADOR]); // preset
const administradorOVendedor = verificarRol([ROLES.ADMINISTRADOR, ROLES.VENDEDOR]);
```

16.6 servidor/controllers/ventaControlador.js — transacción ACID

El bloque más crítico del sistema (resumido):

```
// 1. Anti-fraude: el total se recalcula con los datos del servidor
const subtotalCalculado = items.reduce((s, i) => s + i.cantidad * i.precio_unitario, 0);
if (Math.abs((subtotalCalculado - descuento) - total) > 1)
  return res.status(400).json({ codigo: 'TOTAL_INVALIDO' });

connection = await db.getConnection();
await connection.beginTransaction(); // 2. nada es definitivo aún

// 3. Bloqueo de filas: dos cajas no pueden vender el último ejemplar a la vez
await connection.query('SELECT stock_actual FROM mdc_libros WHERE id = ? FOR UPDATE', [...]);

const [venta] = await connection.query('INSERT INTO mdc_ventas ...'); // 4. cabecera
for (const item of items) { // 5. detalle + stock
  await connection.query('INSERT INTO mdc_detalle_ventas ...');
  await connection.query('UPDATE mdc_libros SET stock_actual = stock_actual - ? WHERE id = ?',
}
await connection.commit(); // 6. todo o...
// catch: await connection.rollback() → ...nada (atomicidad)
// finally: connection.release() → la conexión vuelve al pool
```

16.7 cliente/src/context/AuthContext.jsx — sesión y RBAC en la interfaz

```
export const PERMISOS = {
  [ROLES.ADMINISTRADOR]: { verDashboard: true, crearLibro: true, ... },
  [ROLES.VENDEDOR]: { verDashboard: false, crearLibro: false,
    registrarVenta: true, crearCliente: true, ... }
};

const tienePermiso = (permiso) => PERMISOS[usuario.rol_id]?.[permiso] === true;
// Uso en componentes: {tienePermiso('crearLibro')} && <BotonCrear />
```

La sesión (usuario + token) persiste en `localStorage` y se restaura al recargar. Esta matriz solo controla la **interfaz**: la seguridad real la imponen los middlewares del backend. Los guards `RutaProtegida` (¿hay sesión?) y `RutaProtegidaPorRol` (¿tiene el permiso?) envuelven cada ruta en `App.jsx`.

16.8 cliente/src/services/api.js — interceptores de Axios

```
api.interceptors.request.use((config) => { // ANTES de cada petición
  const token = localStorage.getItem('token_sgi');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(r => r, (error) => { // DESPUÉS de cada respuesta
  if (error.response?.status === 401) { // sesión vencida/inválida
    localStorage.clear();
    window.location.href = '/acceso'; // logout global automático
  }
  return Promise.reject(error);
});
```

17. Pruebas automatizadas

El backend tiene **8 suites** con Jest + Supertest que ejercitan la API real (`cd servidor && npm test`). Al importar `app.js` (y no `index.js`) las pruebas corren sin abrir puertos.

Suite	Qué verifica
<code>pruebas/auth.test.js</code>	Login correcto/incorrecto, validación del JWT, rechazo de rutas protegidas sin token
<code>pruebas/libros.test.js</code>	CRUD de libros, validaciones (título vacío, precio negativo), protección sin token
<code>pruebas/movimientos.test.js</code>	Tipos ENTRADA/SALIDA, cantidad positiva, proveedor obligatorio en entradas, seguridad
<code>pruebas/ventas.test.js</code>	Creación de ventas, carrito vacío, detección de total manipulado (anti-fraude)
<code>pruebas/clientes.test.js</code>	Listado/creación de clientes y protección sin token
<code>pruebas/usuarios.test.js</code>	CRUD de usuarios, RBAC, cambio de contraseña, que password_hash nunca se exponga
<code>pruebas/catalogos.test.js</code>	CRUD de autores y categorías, permisos admin vs. vendedor
<code>pruebas/dashboard.test.js</code>	Acceso exclusivo del admin y estructura del JSON de estadísticas

18. Despliegue en producción

Servicio	URL / Host	Detalle
Frontend	<code>https://sgi-libreria-el-saber-production.up.railway.app</code>	Build de Vite servido con <code>serve -s dist -l \$PORT</code> (SPA routing)
Backend	<code>https://friendly-kindness-production-06fe.up.railway.app</code>	Root Directory <code>servidor</code> · <code>node index.js</code> · puerto inyectado 8080 · healthcheck <code>/</code>
Base de datos	<code>autorack.proxy.rlwy.net:39238</code>	Plugin MySQL de Railway — BD <code>inventario_libreria</code> ; variables de conexión inyectadas
Imágenes	Cloudinary (carpeta <code>sgi_portadas</code>)	Redimensión automática 400×600, formatos JPG/PNG/WebP

- **trust proxy:** obligatorio en Railway para que el rate limiting lea la IP real (cabecera `X-Forwarded-For`).
- **Graceful shutdown:** `index.js` captura SIGTERM/SIGINT, cierra el servidor HTTP y el pool de MySQL (evita «Too many connections») con cierre forzado a los 10 s.
- **PM2** (`pm2.config.js`) queda como gestor alternativo: reinicio automático y límite de 200 MB de RAM.
- **Secretos:** JWT y Cloudinary se configuran en el panel de Railway; nunca en el repositorio.

19. Estructura de carpetas del proyecto

```
proyecto-inventario/
├─ base_datos/
│   └─ sgi_libreria_completo.sql      # BD completa: tablas, índices, vistas y datos semilla
├─ cliente/                          # FRONTEND – React 19 + Vite 7
│   ├── index.html                  # Único HTML: React monta en #root
│   ├── vite.config.js              # build + code-splitting (vendor/charts/exportar)
│   ├── railway.json                # despliegue del frontend en Railway
│   └─ src/
│       ├── main.jsx                # arranque: estilos + createRoot(<App />)
│       ├── App.jsx                 # rutas + guards de seguridad
│       ├── pages/                  # 11 páginas funcionales + 4 de documentación SENA
│       ├── components/             # navbar, layout, modal contraseña, 2 guards
│       ├── context/AuthContext.jsx # sesión + matriz PERMISOS (RBAC de UI)
│       ├── services/api.js          # Axios + interceptores JWT/401
│       ├── hooks/usePaginacion.js   # paginación reutilizable
│       └─ styles/custom-theme.css   # tema Evergreen & Magenta Bloom
├─ servidor/                         # BACKEND – Express 5
│   ├── index.js                    # arranque + graceful shutdown
│   ├── app.js                      # middlewares + rutas + health check
│   ├── config/db.js                # pool MySQL (mysql2/promise)
│   ├── routes/                     # 10 enrutadores (uno por módulo)
│   ├── controllers/                # 10 controladores (lógica de negocio)
│   ├── middlewares/                # verificarToken · verificarRol · ratelimiter ·
│   │                               # uploadImagen · validarParametroId · errorHandler
│   ├── utils/paginacion.js          # helpers de paginación del servidor
│   ├── pruebas/                    # 8 suites Jest + Supertest
│   ├── pm2.config.js                # gestor de procesos alternativo
│   └─ railway.json                  # despliegue del backend en Railway
├─ manuales/                        # Manual de Usuario · Instalación · Técnico
└─ README.md                        # documentación general del repositorio
```

20. Convenciones y elementos avanzados de la base de datos

20.1 Convenciones del proyecto

- **Prefijo** `mdc_` en todas las tablas: evita colisiones de nombres en hosting compartido e identifica las tablas del proyecto.
- **Código y dominios en español** (controladores, rutas, mensajes): coherente con el usuario final y el contexto académico.
- **DECIMAL(10,2)** para dinero (nunca FLOAT, que introduce errores de redondeo).
- **Borrado lógico** en usuarios (campo `estado`) y **anulación** en ventas (campo `estado`): el historial nunca se destruye.

- **DRY:** validación de IDs, paginación, RBAC y manejo de errores centralizados en middlewares/utilidades reutilizables.

20.2 Vistas SQL incluidas

Vista	Qué entrega
v_libros_stock_bajo	Libros con <code>stock_actual ≤ stock_minimo</code> , con autor y categoría — alimenta las alertas
v_ventas_hoy	Conteo, ingresos y promedio de las ventas completadas del día (COALESCE evita nulos)
v_catalogo_libros	Catálogo completo con el campo calculado <code>estado_stock</code> (CASE: Agotado / Stock Bajo / Disponible)

20.3 Índices de rendimiento

Índice	Tabla (columnas)	Acelera
idx_ventas_fecha / _cliente / _estado	mdc_ventas (fecha_venta · cliente_id · estado)	Dashboard e historial con filtros
idx_detalle_venta / _libro	mdc_detalle_ventas (venta_id · libro_id)	JOIN de detalles y «más vendidos»
idx_libros_stock	mdc_libros (stock_actual, stock_minimo) — compuesto	Alertas de stock bajo
idx_libros_titulo	mdc_libros (titulo)	Búsqueda por título
idx_mov_libro / _fecha / _proveedor	mdc_movimientos	Consultas del Kardex
idx_clientes_nombre	mdc_clientes (nombre_completo)	Buscador de clientes

Fin del Manual Técnico. SGI Librería El Saber · Proyecto de grado ADSO — Ficha 3118323 · SENA Regional Tolima · 2026. Los tres manuales (Usuario, Instalación y Técnico) conforman la documentación de sustentación del proyecto.